Software Supply Chain Security

Jingyue Li

20 March 2026

Agenda

- Software supply chain
- Software supply chain threats
- Software supply chain security countermeasure strategies and technologies

Software supply chain (SCC)

*

- Organization's use of externally supplied software (open source or commercially purchased) in products

* https://query.prod.cms.rt.microsoft.com/cms/api/am/binary/RWT0NI

The components of a link in SSC

- Upstream
  - What you depend on
- Downstream
  - What depends on you
- Artifacts
  - Code, infrastructure
- Operations
  - Fetching, scanning, deployment
- Actors
  - Modifying artifacts and operations
- Connections
  - How these components come together

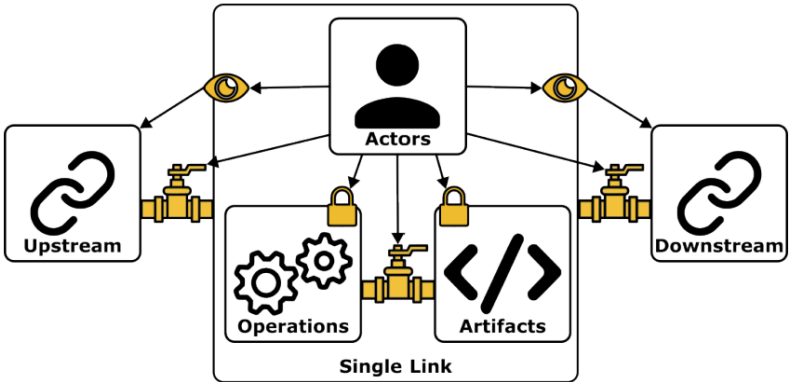


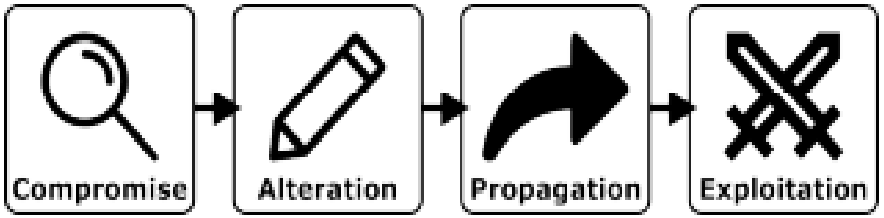
Illustration of a single link in the SSC

Security in software supply chain

- Increased dependency, complexity, and opaqueness
  - More opportunities for malicious actors to exploit vulnerabilities
- Like a physical chain, a weak link can compromise the whole chain

Software supply chain attack

- Compromise
- Alteration
- Propagation
- Exploitation

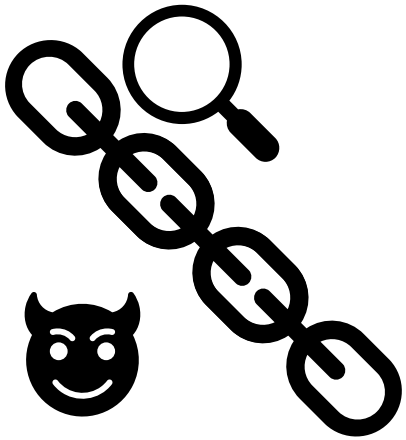


Stages of SSC attack[1]

Compromise

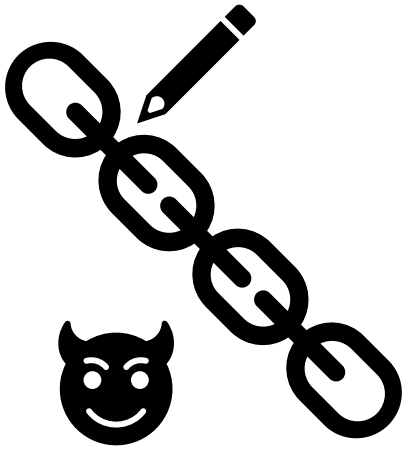
- Find existing vulnerability to exploit SSC
  - Identifying the weak link

- Typical flaws:
  - Authentication failure
  - Design of software
  - Build infrastructure
  - Vulnerability introduced by dependency



Alteration

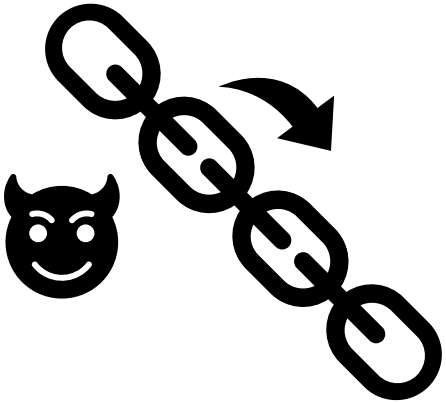
- Alteration of the link in the SSC
  - Making a malicious change upstream
- **Compromise** allows access
- Adversary makes malicious alteration



Propagation

- Propagate malicious alteration to SSC
  - Spread malicious **alteration** from upstream to downstream

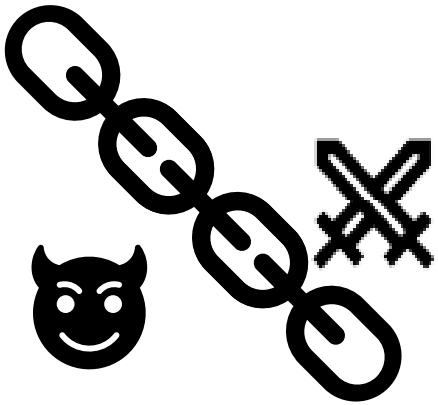
- Method of propagation
  - Pushing updates to users directly
  - Publishing updated software online
    - Centralized repositories



Exploitation

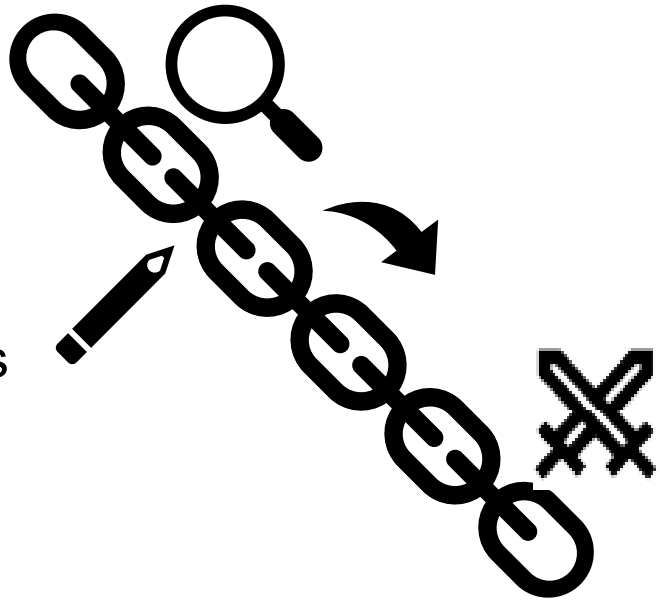
- Exploit vulnerability downstream in SSC
  - Utilizing **alteration** that has been **propagated** to the end-user

- Fulfillment of adversarial goal
  - Confidentiality
  - Integrity
  - Availability
  - Privacy



An example of software supply chain security attack [2]

- SSC attack targeting SolarWinds in 2020



- Gained unauthorized access to the SolarWinds network (compromise)
- Inserted malicious code into the SolarWinds software build process (alteration)
- SolarWinds unknowingly distributed the compromised update to its customers (propagation)
- The malicious code enabled the attacker to access customer systems once the update was installed (exploitation)

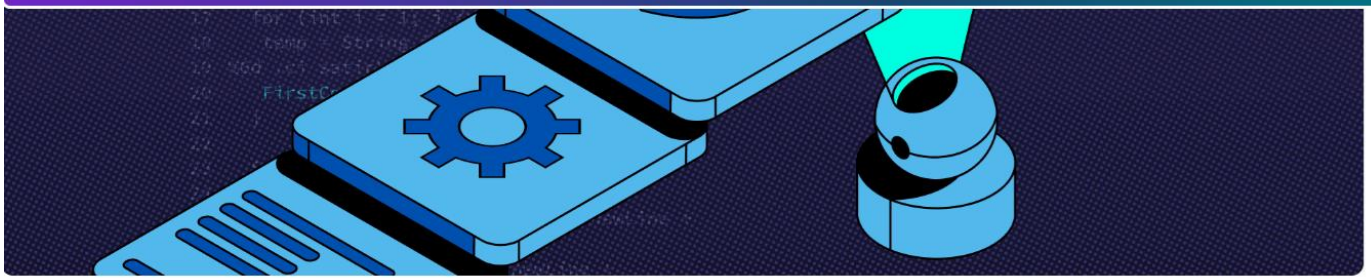
Difference between supply chain attack and vulnerable components

- A06-2021 (OWASP top 10): Vulnerable and Outdated Components
  - Could be the consequence of careless or unintended use/integration of vulnerable components by downstream users.

- Supply chain attacks always have malicious attackers in the loop who purposely inject vulnerabilities and plan to exploit them in the future.

More examples of software supply chain attacks

Search Blog



LinkedIn

Facebook

Bluesky

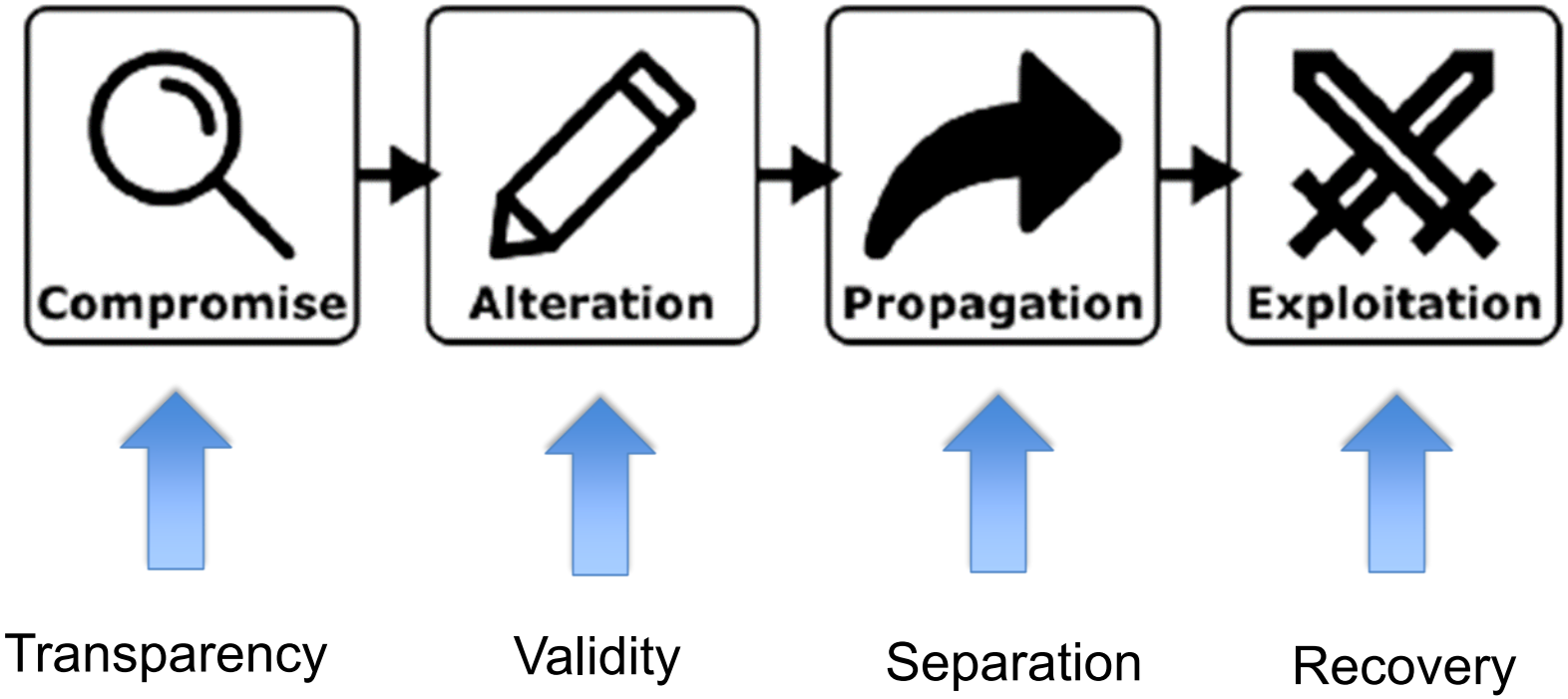
by Pallavi Kalapatapu Published on 04/26/2023 Last updated on 12/23/2025

Top 15 software supply chain attacks: Case studies

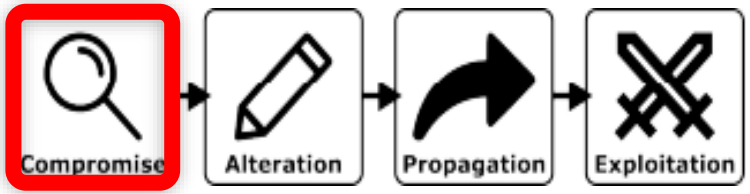
We've talked before about why software supply chains cannot be ignored and shared a primer on supply chains, what software supply chains are, why they can lead to security breaches, and how to safeguard them.

More can be found on https://outshift.cisco.com/blog/top-10-supply-chain-attacks

Countermeasure strategies

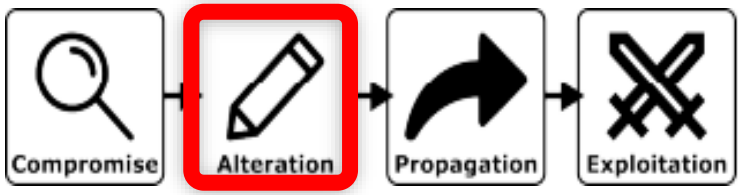


Transparency



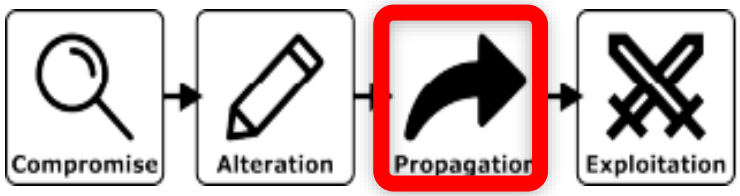
- Transparency builds trust and security.
- Enables perfect vision of all actors, operations, and artifacts across the supply chain.
- Allow supply chain managers to identify link weaknesses before they are compromised.
- By identifying weaknesses first, managers prevent attackers from completing the first stage.

Validity



- By maintaining
  - integrity of artifacts
  - perfect integrity of operations
  - authentication of actors
- No unauthorized changes can be made to the supply chain.

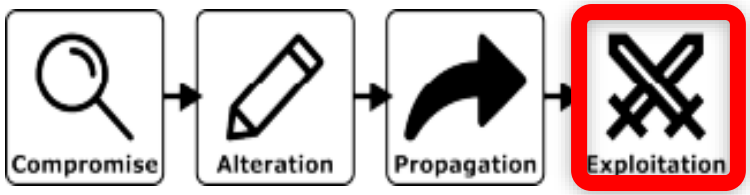
Separation



- Compartmentalize and moderate interactions between entities.

- Connections between artifacts, operations, and actors are managed so that malicious changes cannot affect other supply chain components.

Recovery



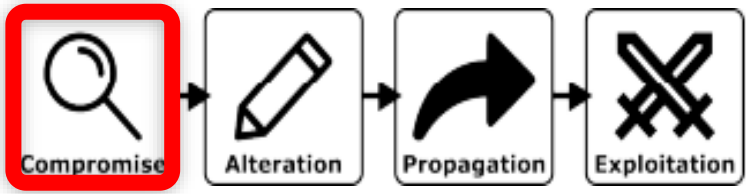
- Response plan
- Minimize impacts

Countermeasure techniques

Techniques	Transparency			Validity			Separation		
	Artifacts	Operations	Actors	Artifacts	Operations	Actors	Artifacts	Operations	Actors
SBOM	✓	✓							
npm-audit [55]	✓			✓					
Code scanning [1]	✓			✓					
Dependabot features [29]	✓			✓					
GitHub Actions [28]		✓		✓	✓			✓	
Git Commit Signing [27]			✓	✓					
Scope [54]				✓			✓		✓
Multi-Factor Authentication						✓			
In-toto [73]	✓	✓		✓	✓			✓	✓
Containerization							✓	✓	✓
Version Locking							✓		
Sigstore [51]	✓	✓	✓	✓	✓				
Mirroring and Proxies [53]	✓			✓			✓	✓	

From [1]

Software Bill of Materials (SBOM)



- A SBOM is a nested inventory, a list of ingredients that comprise software components.
- “Minimum elements” for an SBOM*

Minimum Elements	
Data Fields	Document baseline information about each component that should be tracked: Supplier, Component Name, Version of the Component, Other Unique Identifiers, Dependency Relationship, Author of SBOM Data, and Timestamp.
Automation Support	Support automation, including via automatic generation and machine-readability to allow for scaling across the software ecosystem. Data formats used to generate and consume SBOMs include SPDX, CycloneDX, and SWID tags.
Practices and Processes	Define the operations of SBOM requests, generation and use including: Frequency, Depth, Known Unknowns, Distribution and Delivery, Access Control, and Accommodation of Mistakes.

* https://www.ntia.doc.gov/report/2021/minimum-elements-software-bill-materials-sbom

An example of SBOM

- In Software Package Data eXchange (SPDX) format

Description *

```
content
├── build
│   └── hello
└── src
    ├── Makefile
    └── hello.c
```

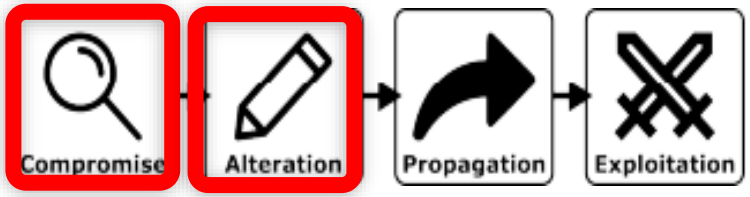
One C source file with a simple "hello world" program, compiled into a single binary with no dependencies via a Makefile. (Assumed dependencies such as the operating system kernel, C standard library, etc. are not addressed here.)

* https://github.com/swinslow/spdx-examples/blob/master/example1/spdx/example1.spdx

```
SPDXVersion: SPDX-2.2
DataLicense: CC0-1.0
SPDXID: SPDXRef-DOCUMENT
DocumentName: hello
DocumentNamespace: https://swinslow.net/spdx-examples/example1/hello-v3
Creator: Person: Steve Winslow (steve@swinslow.net)
Creator: Tool: github.com/spdx/tools-golang/builder
Creator: Tool: github.com/spdx/tools-golang/idsearcher
Created: 2021-08-26T01:46:00Z

##### Package: hello

PackageName: hello
SPDXID: SPDXRef-Package-hello
PackageDownloadLocation: git+https://github.com/swinslow/spdx-examples.git#example1/content
FilesAnalyzed: true
PackageVerificationCode: 9d20237bb72087e87069f96afb41c6ca2fa2a342
PackageLicenseConcluded: GPL-3.0-or-later
PackageLicenseInfoFromFiles: GPL-3.0-or-later
PackageLicenseDeclared: GPL-3.0-or-later
PackageCopyrightText: NOASSERTION

Relationship: SPDXRef-DOCUMENT DESCRIBES SPDXRef-Package-hello
```

```
25      FileName: /build/hello
26      SPDXID: SPDXRef-hello-binary
27      FileType: BINARY
28      FileChecksum: SHA1: 20291a81ef065ff891b537b64d4fdccaf6f5ac02
29      FileChecksum: SHA256: 83a33ff09648bb5fc5272baca88cf2b59fd81ac4cc6817b86998136af368708e
30      FileChecksum: MD5: 08a12c966d776864cc1eb41fd03c3c3d
31      LicenseConcluded: GPL-3.0-or-later
32      LicenseInfoInFile: NOASSERTION
33      FileCopyrightText: NOASSERTION
34
35      FileName: /src/Makefile
36      SPDXID: SPDXRef-Makefile
37      FileType: SOURCE
38      FileChecksum: SHA1: 69a2e85696fff1865c3f0686d6c3824b59915c80
39      FileChecksum: SHA256: 5da19033ba058e322e21c90e6d6d859c90b1b544e7840859c12cae5da005e79c
40      FileChecksum: MD5: 559424589a4f3f75fd542810473d8bc1
41      LicenseConcluded: GPL-3.0-or-later
42      LicenseInfoInFile: GPL-3.0-or-later
43      FileCopyrightText: NOASSERTION
44
45      FileName: /src/hello.c
46      SPDXID: SPDXRef-hello-src
47      FileType: SOURCE
48      FileChecksum: SHA1: 20862a6d08391d07d09344029533ec644fac6b21
49      FileChecksum: SHA256: b4e5ca56d1f9110ca94ed0bf4e6d9ac11c2186eb7cd95159c6fdb50e8db5a823
50      FileChecksum: MD5: 935054fe899ca782e11003bbae5e166c
```

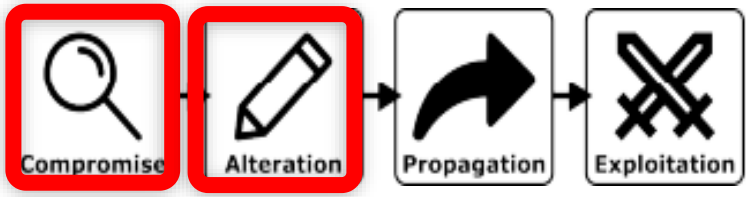
NPM audit



- It automatically checks all your dependencies and its dependency tree for packages that are vulnerable to security flaws.
- Command: npm audit

Moderate	Prototype pollution
Package	hoek
Patched in	> 4.2.0 < 5.0.0 || >= 5.0.3
Dependency of	numbat-emitter
Path	numbat-emitter > request > hawk > boom > hoek
More info	https://nodesecurity.io/advisories/566

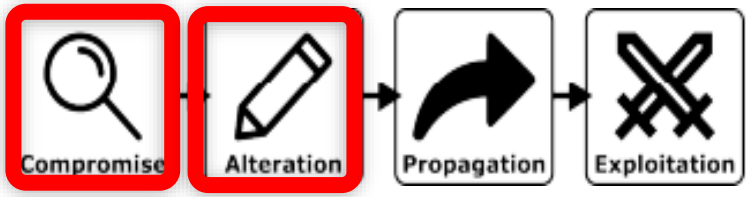
Dependabot



- Discovers insecure dependencies in your project.
- When GitHub detects a vulnerable dependency in the default branch, dependabot creates a pull request to fix it.

- Pull request will upgrade the dependency to the minimum possible secure version needed to avoid the vulnerability.

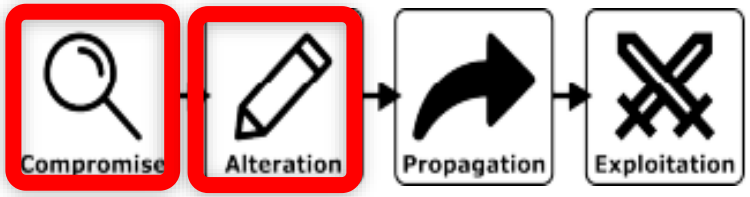
GitHub Actions



- Threat model
  - The attack can modify the build process.

- Countermeasures
  - The build steps should be precise and repeatable.
  - You know exactly what was running during the build process.
  - Ensure each build starts in a new environment to minimize the risk of attackers maintaining persistence within the build system.

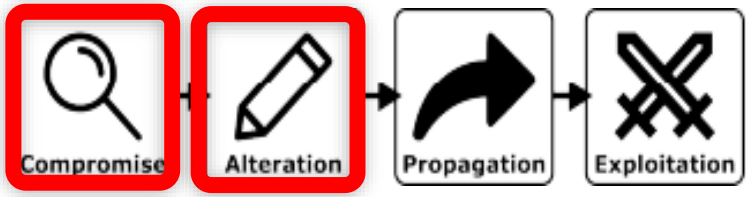
Git Commit Signing



- Transparency (Actors) and Validity (Artifacts)
- Generate a private and public key pair.
- Use your private key to sign your commit.
- Use the public key to verify the author of a commit.

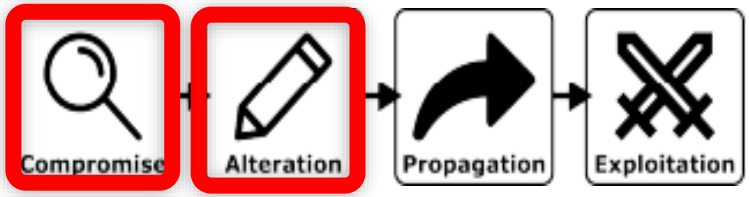
* https://git-scm.com/book/en/v2/Git-Tools-Signing-Your-Work

Sigstore (1/2)



- Making software signing part of an invisible and ubiquitous infrastructure.
- Using existing identity providers to issue short-lived certificates for individual package signing workflows.
- **Identity-based (OIDC):** Instead of verifying a key, the system verifies the user's identity via OpenID Connect (OIDC)—for example, using a GitHub, Google, or Microsoft account.
- Users can sign using ephemeral keys (“keyless signing”), which allows developers to sign packages without managing their cryptographic material.

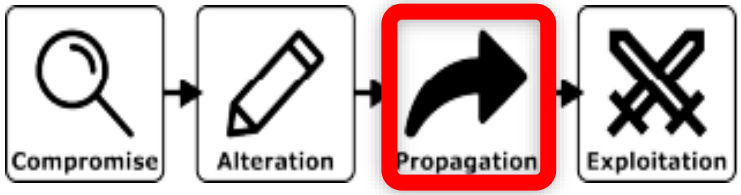
Sigstore (2/2)



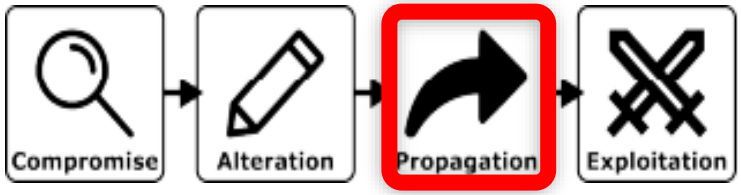
- **Transparency Log:** Because the keys are temporary, the signature is recorded in a public, tamper-resistant log (like Sigstore's Rekor). This log provides a permanent record that the signature was valid at the time of creation.

- Hosts more than two million (as of April 2022) different package signatures over more than 450 GitHub repositories.

Containerization



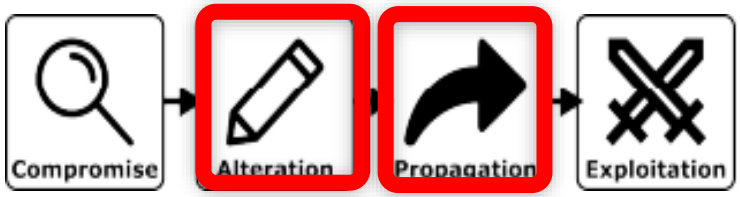
- Threat model
  - Attackers can propagate the attack or attack consequence via unintended connections.
- Countermeasure
  - Remove unnecessary connections and separate internal operations, artifacts, and actors.



Version Locking

- Threat model
  - Malicious changes upstream may be automatically propagated to downstream links.
- Countermeasure
  - Version locking ensures that a link includes a particular version of an upstream component.
  - However, it relies on actors to accurately set and manage version numbers.

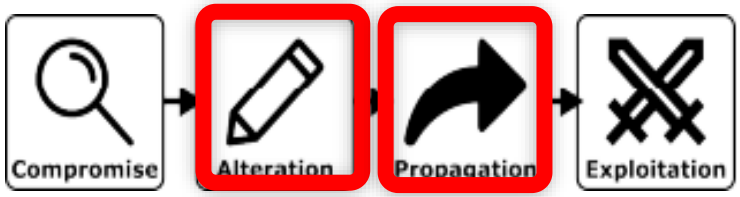
Scope



- Threat model
  - Dependency confusion risks where an internal package name is claimed by an attacker on the public registry.
- Countermeasure
  - Restricting the package's namespace to an organization or user using scope (@somescope/somepackagename).
  - Scopes can be associated with a given registry, which ensures that all requests for packages under the scope will be routed to the given registry.

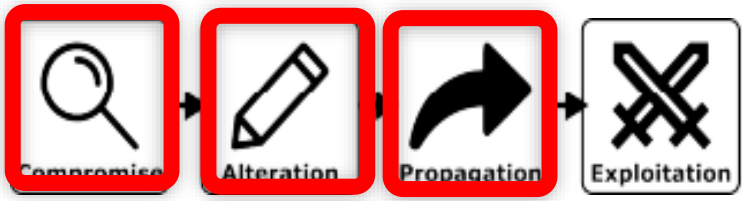
```
$ npm login --scope=@myorg
    --registry=http://registry.myorg.com
```

Proxy



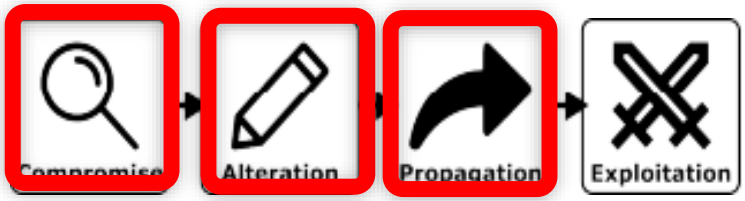
- Threat model
  - An attacker might publish a malicious package to the public repository with the same name as a package hosted on a private registry but with a higher semantic version.
  - If a custom setting for an internal registry is omitted, the package manager would default to the public registry and download the latest (malicious) packages from there.
- Countermeasure
  - Configuring the proxy never to allow an upstream request to the public registries protects against fetching arbitrary packages in place of the legitimate package.

Mirroring (1/2)



- Threat model
  - The package manager may download the malicious packages from the public registry.
- Countermeasure
  - Organizations create private package feeds to mitigate the risk of pulling dependencies from public sources.

Mirroring (2/2)



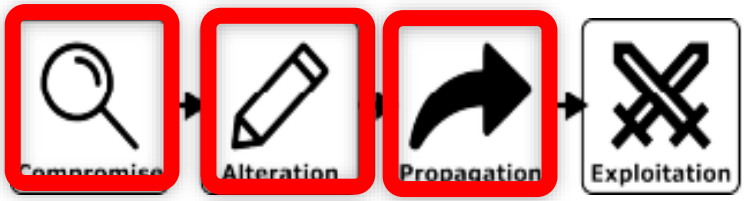
*

Maven example

```
<settings>
  ...
  <mirrors>
    <mirror>
      <id>other-mirror</id>
      <name>Other Mirror Repository</name>
      <url>https://other-mirror.repo.other-company.com/maven2</url>
      <mirrorOf>central</mirrorOf>
    </mirror>
  </mirrors>
  ...
</settings>
```

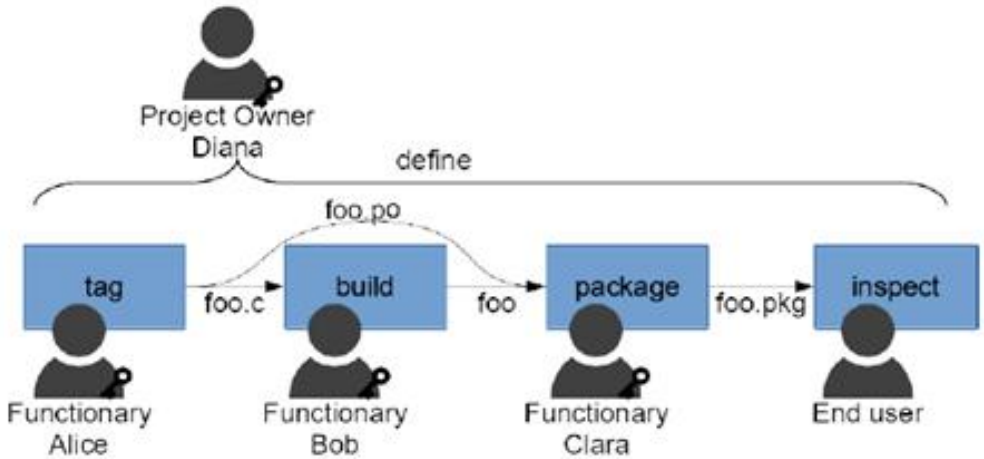
* https://maven.apache.org/guides/mini/guide-mirror-settings.html#using-a-single-repository

In toto (Latin for “as a whole”) (1/3)



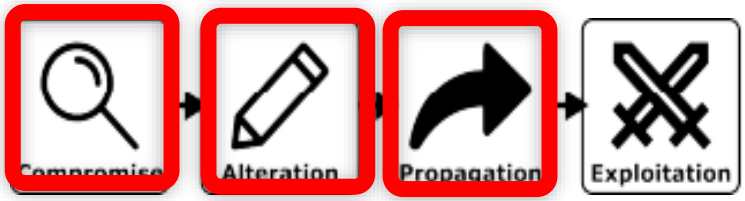
- One of the key element: Layout

Layout is a recipe that identifies which steps will be performed, by whom, and in what order



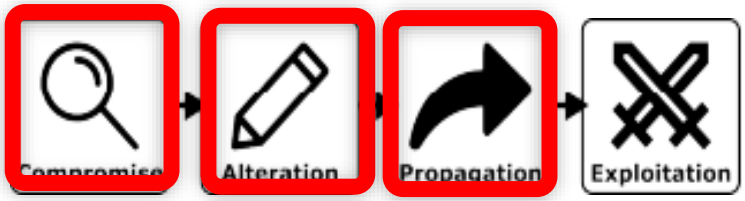
From [3]

In toto (2/3)



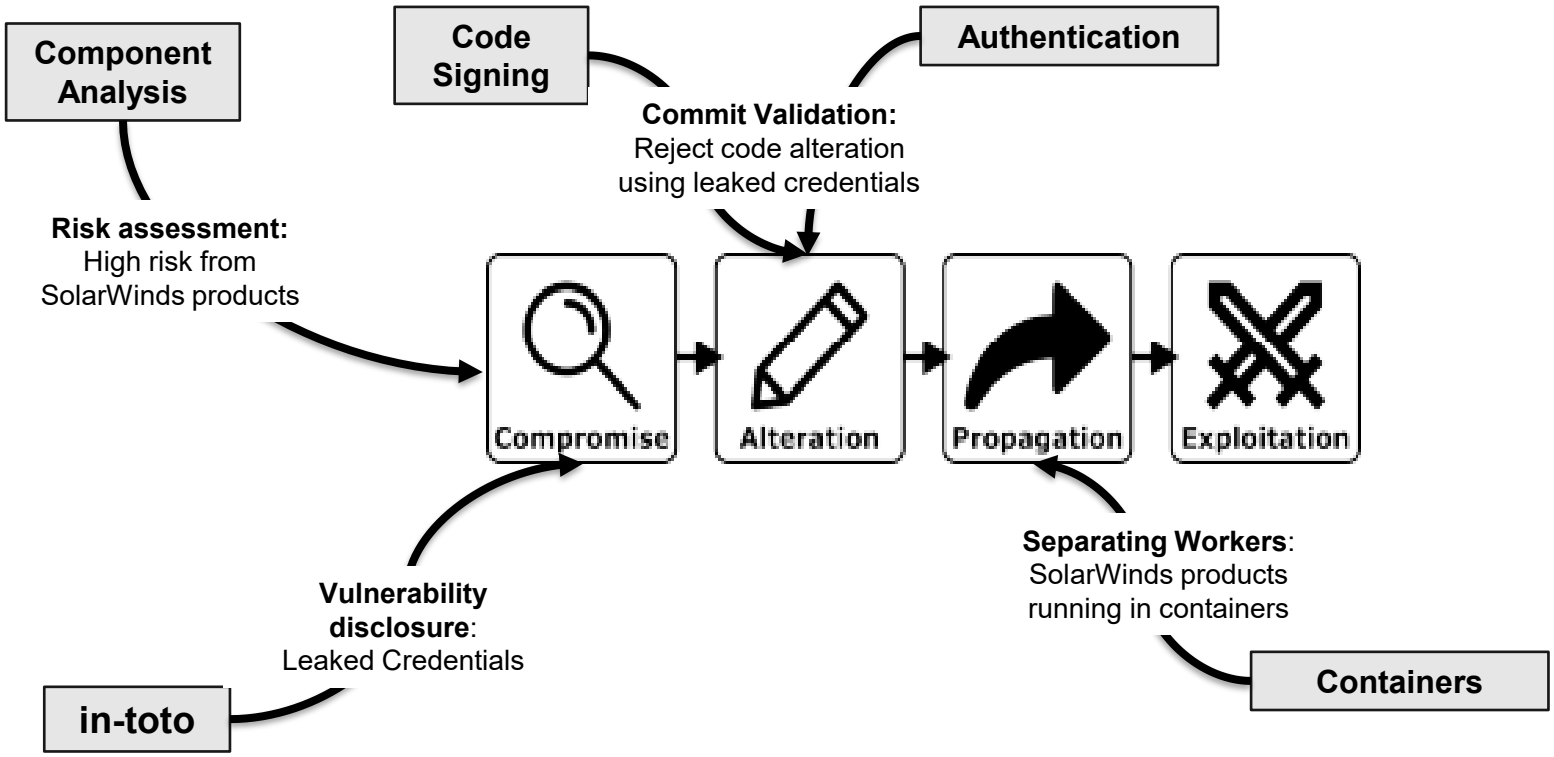
- Another key element: Link metadata
  - Each link serves as a statement that a given step was carried out by whom, where, and when.
  - Functionaries executing a step within the supply chain must share information about these links.
  - One-to-one relationship between the step definitions in the supply chain layout and the link metadata.
  - The intended entity must cryptographically sign link metadata
  - Sharing such information ensures no artifacts are altered in transit.

In toto (3/3)



- The third key element: the delivered product
  - To verify the delivered product, the end user will utilize the supply chain layout and its corresponding pieces of link metadata.
  - The end user will use the link metadata to verify that the software provided has not been tampered with and that all the steps were performed as the project owner intended.

Security properties against SOLARBURST



Research way forward

- Most approaches focus on managing artifacts.
- More approaches are needed to focus on operations and actors.
- More empirical studies on using the proposed approaches in practice.

Summary

- Supply chain attacks
  - **Compromise**
  - **Alteration**
  - **Propagation**
  - **Exploitation**

- Countermeasure strategies
  - **Transparency**
  - **Validity**
  - **Separation**
  - **Recovery**

References

- [1] Okafor, Chinenye, et al. "Sok: Analysis of software supply chain security by establishing secure design properties." *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. 2022.
- [2] https://www.microsoft.com/en-us/msrc/blog/2020/12/customer-guidance-on-recent-nation-state-cyber-attacks.
- [3] Torres-Arias et al. In-toto: providing farm-to-table guarantees for bits and bytes. In Proceedings of the 28th USENIX Conference on Security Symposium (SEC'19).